

Seller and Inventory Management System

DBMS / MySQL Practical Assignment

Name	Mohanraj S
Roll No	TU6253210211033
Week	3

1. Create the Database

```
CREATE DATABASE SellerInventoryDB;  
USE SellerInventoryDB;
```

2. Create the Seller Table

```
CREATE TABLE Seller (  
  SellerID INT PRIMARY KEY AUTO_INCREMENT,  
  SellerName VARCHAR(100) NOT NULL,  
  Email VARCHAR(100) UNIQUE,  
  Phone VARCHAR(15),  
  Address VARCHAR(200)  
);
```

Insert Seller Data

```
INSERT INTO Seller (SellerName, Email, Phone, Address)  
VALUES  
( 'Arun Electronics', 'arun@gmail.com', '9876543210', 'Chennai'),  
( 'Kumar Mobiles', 'kumar@gmail.com', '9876543211', 'Madurai'),  
( 'Tech World', 'techworld@gmail.com', '9876543212', 'Coimbatore');
```

3. Create the Product Table

```
CREATE TABLE Product (  
  ProductID INT PRIMARY KEY AUTO_INCREMENT,  
  ProductName VARCHAR(100) NOT NULL,  
  Category VARCHAR(50),  
  Price DECIMAL(10,2) NOT NULL  
);
```

Insert Product Data

```
INSERT INTO Product (ProductName, Category, Price)  
VALUES  
( 'Laptop', 'Electronics', 55000.00),  
( 'Smartphone', 'Electronics', 25000.00),  
( 'Keyboard', 'Accessories', 1500.00),  
( 'Mouse', 'Accessories', 800.00),  
( 'Headphones', 'Accessories', 2000.00);
```

4. Create the Inventory Table

```
CREATE TABLE Inventory (
  InventoryID INT PRIMARY KEY AUTO_INCREMENT,
  SellerID INT,
  ProductID INT,
  StockQuantity INT NOT NULL,
  StockStatus VARCHAR(20),
  FOREIGN KEY (SellerID) REFERENCES Seller(SellerID),
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
```

Insert Inventory Data

```
INSERT INTO Inventory
(SellerID, ProductID, StockQuantity, StockStatus)
VALUES
(1, 1, 10, 'Available'), (1, 3, 25, 'Available'),
(2, 2, 0, 'Unavailable'),
(2, 4, 15, 'Available'),
(3, 5, 0, 'Unavailable');
```

5. Display All Sellers

```
SELECT * FROM Seller;
```

6. Display All Products

```
SELECT * FROM Product;
```

7. Display Inventory

```
SELECT * FROM Inventory;
```

8. Seller–Product–Inventory Relationship

```
SELECT
  s.SellerName, p.ProductName, p.Category, p.Price,
  i.StockQuantity, i.StockStatus
FROM Inventory i
JOIN Seller s ON i.SellerID = s.SellerID
JOIN Product p ON i.ProductID = p.ProductID;
```

9. Find Available Products

```
SELECT
  p.ProductName, s.SellerName, i.StockQuantity
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID
JOIN Seller s ON i.SellerID = s.SellerID
WHERE i.StockQuantity > 0;
```

10. Find Unavailable Products

```
SELECT
```

```
    p.ProductName, s.SellerName, i.StockQuantity
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID
JOIN Seller s ON i.SellerID = s.SellerID
WHERE i.StockQuantity = 0;
```

11. Update Stock

```
UPDATE Inventory
SET StockQuantity = 20, StockStatus = 'Available'
WHERE ProductID = 1;
```

12. Make a Product Unavailable

```
UPDATE Inventory
SET StockQuantity = 0, StockStatus = 'Unavailable'
WHERE ProductID = 2;
```

13. Inventory Status Report

```
SELECT
    s.SellerName, p.ProductName, i.StockQuantity, i.StockStatus
FROM Inventory i
JOIN Seller s ON i.SellerID = s.SellerID
JOIN Product p ON i.ProductID = p.ProductID
ORDER BY i.StockQuantity DESC;
```

14. Count Available and Unavailable Products

```
SELECT
    StockStatus, COUNT(*) AS TotalProducts
FROM Inventory
GROUP BY StockStatus;
```

15. Find Low Stock Products

```
SELECT
    p.ProductName, s.SellerName, i.StockQuantity
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID
JOIN Seller s ON i.SellerID = s.SellerID
WHERE i.StockQuantity <= 5;
```

16. Complete SQL Code

```
CREATE DATABASE SellerInventoryDB;
USE SellerInventoryDB;

CREATE TABLE Seller (
    SellerID INT PRIMARY KEY AUTO_INCREMENT,
    SellerName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
```

```

    Phone VARCHAR(15),
    Address VARCHAR(200)
);

CREATE TABLE Product (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductName VARCHAR(100) NOT NULL,
    Category VARCHAR(50),
    Price DECIMAL(10,2) NOT NULL
);

CREATE TABLE Inventory (
    InventoryID INT PRIMARY KEY AUTO_INCREMENT,
    SellerID INT,
    ProductID INT,
    StockQuantity INT NOT NULL,
    StockStatus VARCHAR(20),
    FOREIGN KEY (SellerID) REFERENCES Seller(SellerID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);

INSERT INTO Seller (SellerName, Email, Phone, Address)
VALUES
('Arun Electronics', 'arun@gmail.com', '9876543210', 'Chennai'),
('Kumar Mobiles', 'kumar@gmail.com', '9876543211', 'Madurai'),
('Tech World', 'techworld@gmail.com', '9876543212', 'Coimbatore');

INSERT INTO Product (ProductName, Category, Price)
VALUES
('Laptop', 'Electronics', 55000.00),
('Smartphone', 'Electronics', 25000.00),
('Keyboard', 'Accessories', 1500.00),
('Mouse', 'Accessories', 800.00),
('Headphones', 'Accessories', 2000.00);

INSERT INTO Inventory
(SellerID, ProductID, StockQuantity, StockStatus)
VALUES
(1, 1, 10, 'Available'),
(1, 3, 25, 'Available'),
(2, 2, 0, 'Unavailable'),
(2, 4, 15, 'Available'),
(3, 5, 0, 'Unavailable');

SELECT s.SellerName, p.ProductName, p.Category, p.Price,
       i.StockQuantity, i.StockStatus
FROM Inventory i
JOIN Seller s ON i.SellerID = s.SellerID
JOIN Product p ON i.ProductID = p.ProductID;

SELECT p.ProductName, s.SellerName, i.StockQuantity
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID
JOIN Seller s ON i.SellerID = s.SellerID
WHERE i.StockQuantity > 0;
SELECT p.ProductName, s.SellerName, i.StockQuantity

```

```
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID
JOIN Seller s ON i.SellerID = s.SellerID
WHERE i.StockQuantity = 0;

SELECT StockStatus, COUNT(*) AS TotalProducts
FROM Inventory
GROUP BY StockStatus;
```

17. Viva / Explanation Points

1. Seller table stores seller details.
2. Product table stores product details.
3. Inventory table connects sellers and products and stores stock information.
4. SellerID is a Foreign Key in the Inventory table.
5. ProductID is a Foreign Key in the Inventory table.
6. StockQuantity stores the number of available units.
7. StockStatus stores whether the product is Available or Unavailable.
8. JOIN is used to combine data from Seller, Product, and Inventory.
9. A separate Inventory table represents the relationship between sellers and products and stores seller-specific stock.
10. Primary Key uniquely identifies each record.
11. Foreign Key creates a relationship between two tables.
12. AUTO_INCREMENT automatically generates IDs.
13. GROUP BY groups records by stock status.
14. COUNT() counts products in each stock-status category.
15. ORDER BY sorts the inventory report by stock quantity.